

CCA-F Exam Prep	Domain 1	Agentic Architecture & Orchestration	27% of exam · 7 task state ments · hi ghest-wei ghted domain	Task statements: 1.1 Agentic Loops · 1.2 Multi-Agent Orchestration · 1.3 Subagent Invocation · 1.4 Workflow Enforcement · 1.5 Agent SDK Hooks · 1.6 Task Decomposition · 1.7 Session State
-----------------------	----------	--	--	--

1.1 Agentic Loops

The core execution cycle of every Claude-based agent

What is an agentic loop?

A **deterministic control flow cycle** — not a prompt trick, retry loop, or chatbot turn. Claude receives a task, calls tools, receives results, reasons, calls more tools, and eventually returns a final answer via the Messages API.

The 4-step lifecycle

- 1

Send request

Call the Messages API with full conversation history: system prompt + prior messages + any tool results from the previous iteration.
- 2

Inspect stop_reason

The **ONLY** authoritative signal. Two relevant values: tool_use (loop continues) and end_turn (loop terminates). Never use anything else.
- 3

If tool_use → execute & append

Run the requested tool(s). Append tool results to conversation history as a new message. Send the updated history back to Claude. Without appending, Claude cannot reason about the new information.
- 4

If end_turn → done

Extract the final text response and present to the user. Loop terminates.

KEY
CONCEPT

stop_reason is deterministic and unambiguous. It is the only reliable loop control signal. Natural language parsing, content-type checks, and iteration caps are all wrong as primary stopping mechanisms.

The 3 Anti-Patterns (exam tests all 3)

EXAM TRAP — ANTI-PATTERN 1	Parsing natural language: checking if Claude said 'I'm done'. Wrong because natural language is ambiguous — 'I've finished the first file' might mean more files remain.
EXAM TRAP — ANTI-PATTERN 2	Iteration caps as primary control: 'stop after 10 loops'. Caps either cut off useful work or waste iterations. Acceptable ONLY as a safety net, never as the primary mechanism.
EXAM TRAP — ANTI-PATTERN 3	Checking content type: response.content[0].type == 'text'. Wrong because Claude can return text alongside a tool_use block. Text presence does not equal completion.

Model-driven vs Pre-configured decisions

Model-driven (flexible) • Claude selects tool based on context • Adapts to edge cases and unexpected situations • Exam favours this for flexibility • Exception: when compliance is required	Pre-configured (deterministic) • Fixed tool sequences / decision trees • Developer hard-codes which tool runs when • Required for financial, security, compliance • Overrides model decisions deterministically
--	---

1.2 Multi-Agent Orchestration

Hub-and-spoke architecture with a coordinator at the centre

Hub-and-spoke architecture

Coordinator (hub) • Receives initial task from user • Decomposes task into subtopics • Delegates to specific subagents • Aggregates and synthesises results • Handles errors uniformly • Routes ALL information — no exceptions	Subagents (spokes) • Handles one specialised task only • Receives instructions from coordinator only • Returns results to coordinator only • Never communicates with other subagents • Context is completely isolated per invocation
---	---

KEY CONCEPT

Cardinal rule: ALL communication flows through the coordinator. Subagents NEVER communicate directly with each other — not for efficiency, not for convenience, not for any reason.

Why centralisation matters — 3 exam-tested benefits

Benefit	What it means
Observability	Every message is logged in one place — full audit trail
Consistent error handling	Coordinator applies uniform recovery policies across all subagents
Controlled information flow	Coordinator decides exactly what context each subagent receives

The Isolation Principle (most commonly misunderstood)

Subagents do **NOT** automatically inherit the coordinator's conversation history. Each subagent invocation starts with only what the coordinator explicitly includes in its prompt. No shared memory between invocations. No access to other subagents' results unless the coordinator explicitly passes them.

EXAM TRAP — CLASSIC EXAM FAILURE

A synthesis agent produces unsourced claims. The web search and document analysis subagents ran correctly. Root cause: the coordinator passed findings **WITHOUT** structured metadata — the synthesis agent had no source information to cite. Always trace failures to their origin.

4 Coordinator Responsibilities (all exam-tested)

- 1

Dynamic subagent selection

Analyse the query and select only the needed subagents. Don't route every query through every subagent — route selectively based on what the task requires.
- 2

Research scope partitioning

Assign distinct subtopics or source types to each agent to minimise overlap and maximise coverage breadth.
- 3

Iterative refinement loops

Evaluate synthesis output for gaps. Re-delegate with targeted queries if coverage is insufficient. Not a single-shot process.

All subagent messages flow through the coordinator for observability and consistent error handling.

The Narrow Decomposition Failure (named exam pattern)

Coordinator decomposes 'impact of AI on creative industries' into only visual arts subtopics. Music, writing, and film are never researched. The web search and synthesis subagents both performed correctly — they only worked with what they were assigned.

Fix: improve coordinator decomposition. Adding more subagents won't help — they'll receive equally narrow assignments.

1.3 Subagent Invocation & Context Passing

The Agent tool, AgentDefinition, and structured metadata

The Agent tool

The specific API mechanism for spawning subagents in the Claude Agent SDK. Renamed from 'Task tool' in v2.1.63. The old name Task still works as a backward-compatible alias.

KEY CONCEPT

Hard requirement: the coordinator's allowedTools must include 'Agent' (or 'Task'). Without this, the coordinator physically cannot spawn subagents. This is a binary gate — not a soft preference.

AgentDefinition — 3 required fields

Field	Purpose
description	Tells the coordinator when to invoke this subagent (used for dynamic selection)
system prompt	The instructions the subagent follows for its specialised role
tool restrictions	Which tools the subagent can access — scoped to its role only

Context Passing — 3 Rules

1

Pass complete findings from prior agents

If the synthesis agent needs web search results AND document analysis output, the coordinator must pass both in full — do not assume the synthesis agent can look them up.

2

Use structured data with metadata

Pass content + source URL + document name + page number. Without metadata, downstream agents cannot attribute claims to sources.

3

Goal-oriented prompts, not procedural

Tell subagents WHAT to achieve, not step-by-step HOW. Goal prompts enable adaptability; procedural ones constrain the subagent.

Parallel Spawning vs Sequential

Parallel (correct)

- Emit multiple Agent tool calls in a single coordinator response
- Independent subagents start simultaneously
- Correct approach for independent tasks
- Exam signal: 'in a single response' or 'simultaneously'

Sequential (anti-pattern for independent tasks)

- One subagent invoked per coordinator turn
- Must wait for each to finish before starting the next
- Anti-pattern for independent tasks
- Introduces unnecessary latency

fork_session vs --resume

fork_session • Creates independent branches from a shared baseline • Use for divergent exploration • Forks don't see each other's results • Example: compare two testing strategies after analysing a codebase	--resume • Continues a specific named session • Use to resume the same line of investigation • Same thread, continued from where it left off • Example: pick up interrupted long-running task
--	---

1.4 Workflow Enforcement & Handoff

Programmatic gates vs prompt-based guidance

The Enforcement Spectrum

Prompt-based guidance (probabilistic) • Instructions in the system prompt • Works ~90–95% of the time • Non-zero failure rate (probabilistic) • Acceptable for low-stakes tasks • Example: 'always verify identity before refunding'	Programmatic enforcement (deterministic) • Code-level prerequisite gates • Physically blocks wrong execution order • Works every time (deterministic) • Required for high-stakes tasks • Example: process_refund cannot execute until get_customer returns verified ID
KEY CONCEPT	Exam decision rule: if a single failure would cause financial loss, security breach, or compliance violation — use programmatic enforcement. No exceptions. Enhanced prompts, few-shot examples, and stronger instructions improve probability but never eliminate the failure rate.

When to use each — decision table

Scenario	Enforcement type	Reason
Process a refund	PROGRAMMATIC	Unverified refund = financial loss
Identity verification before account access	PROGRAMMATIC	Bypass = security breach
AML / regulatory compliance checks	PROGRAMMATIC	Missed check = legal penalty
Response formatting preferences	PROMPT-BASED	Formatting error has no business risk
Style guidelines for output	PROMPT-BASED	Low-stakes, inconsistency acceptable

SubagentStart & SubagentStop hooks

Hook	Fires when	Use cases
SubagentStart	A subagent is spawned via the Agent tool	Rate limiting, logging invocations, validating context before subagent begins
SubagentStop	A subagent finishes and returns results	Validating output schema, stripping sensitive data, logging completion
PreToolUse / PostToolUse	Scoped to a specific subagent's tool calls	Per-subagent policy enforcement (e.g. billing subagent blocks refunds above threshold)

Note: Stop hooks defined in subagent frontmatter auto-convert to SubagentStop events at runtime.

1.5–1.7 Decomposition, Hooks & Session State

Task patterns, attention dilution, and resumption

Task Decomposition — 2 Patterns

Fixed sequential pipeline (prompt chaining) • Steps predetermined, execute in fixed order • Each step feeds the next • Consistent, debuggable, predictable • Cannot adapt to unexpected findings • Best for: code reviews, doc processing, compliance checks	Dynamic adaptive decomposition • Subtasks generated from discoveries at each step • Plan evolves as the agent learns more • Adapts to unexpected complexity • Unpredictable execution time • Best for: legacy exploration, security audits, open-ended research
---	--

Pattern selection — quick reference

Task	Pattern
Multi-file code review	Fixed pipeline
Document processing pipeline	Fixed pipeline
Writing tests for known requirements	Fixed pipeline
Legacy codebase exploration	Dynamic decomposition
Security audit of unknown system	Dynamic decomposition
Debugging an unfamiliar codebase	Dynamic decomposition

Attention Dilution

When a single model call must process too many subtasks simultaneously, attention gets spread thin — shallow coverage of every topic rather than deep analysis of any. **Symptom:** output that touches every topic but doesn't go deep on any. **Fix:** break the task into smaller, more focused subtasks so each call can reason deeply about a narrow problem.

Session State & Resumption (1.7)

Concept	What to know
No automatic memory	Claude has no memory between separate API calls — all state must be explicitly managed by the application
--resume	Continues a named session — same thread, same context, pick up where you left off
fork_session	Branches from a shared baseline for divergent exploration — branches are independent
Interrupted tasks	Persist state externally so long tasks can be resumed without restarting from scratch

All 11 Exam Traps — Domain 1		Memorise these before exam day
EXAM TRAP — LOOP CONTROL	Using content[0].type == 'text' to detect completion — Claude can return text alongside tool_use blocks simultaneously.	
EXAM TRAP — LOOP CONTROL	Using iteration caps as the PRIMARY stopping mechanism — they are a safety net only. Primary signal = stop_reason.	
EXAM TRAP — LOOP CONTROL	Parsing natural language ('I'm done') to stop the loop — ambiguous and unreliable by design.	
EXAM TRAP — LOOP CONTROL	Forcing tool_choice: 'any' to prevent text responses — creates infinite loops when the agent is genuinely finished.	
EXAM TRAP — ORCHESTRATION	Blaming a downstream subagent for coverage gaps when the coordinator's decomposition was too narrow.	

EXAM TRAP — ORCHESTRATION	Assuming subagents share memory or inherit coordinator history — they have completely isolated context per invocation.
EXAM TRAP — ORCHESTRATION	Proposing direct inter-subagent communication as an efficiency improvement — breaks observability and error handling.
EXAM TRAP — ORCHESTRATION	Adding more subagents to fix a narrow decomposition problem — they'll receive equally narrow assignments.
EXAM TRAP — CONTEXT PASSING	When synthesis produces unsourced claims, blaming the synthesis prompt — root cause is missing structured metadata in the coordinator's context pass.
EXAM TRAP — ENFORCEMENT	Choosing 'add stronger system prompt instructions' for financial/security/compliance scenarios — prompt-based guidance is probabilistic, never deterministic.
EXAM TRAP — SDK	Forgetting that allowedTools must include 'Agent' — without it, the coordinator cannot spawn any subagents, period.

Domain 1 complete — 27% of the CCA-F exam. Next: Domain 2 · Tool Design & MCP Integration (18%).